

Report Architettura CMMS 2.0

Questo documento illustra la struttura del software del sistema CMMS 2.0, l'organizzazione dei moduli in un **Monolite Modulare** guidato dall'**Architettura Esagonale** (Hexagonal Architecture / Ports and Adapters), e le connessioni tra le varie componenti.

1. Struttura del Progetto & Moduli

Il codice applicativo core si trova sotto la cartella `modules/`. Ogni modulo è isolato e definisce i propri confini di business, interagendo con gli altri tramite contratti stabili (Ports) o eventi.

MAINTENANCE

Modulo Manutenzione

- **Entità pure:** Asset, WorkOrder, PreventiveSchedule, Technician.
- **Logica:** Gestione dello stato degli asset, creazione e avanzamento ticket di manutenzione, programmazione preventiva e disponibilità calendariale dei tecnici.

INVENTORY

Modulo Inventario

- **Entità pure:** SparePart (ricambi), PurchaseRequest (richieste acquisto).
- **Logica:** Controllo giacenze, alert sotto-scorta e approvazione ordini.

ENERGY

Modulo Energia

- **Entità pure:** Meter (contatori), MeterReading (letture).
- **Logica:** Telemetria dei consumi elettrici, acqua, gas. Calcolo emissioni CO2 stimato ed algoritmi di rilevamento anomalie di consumo.

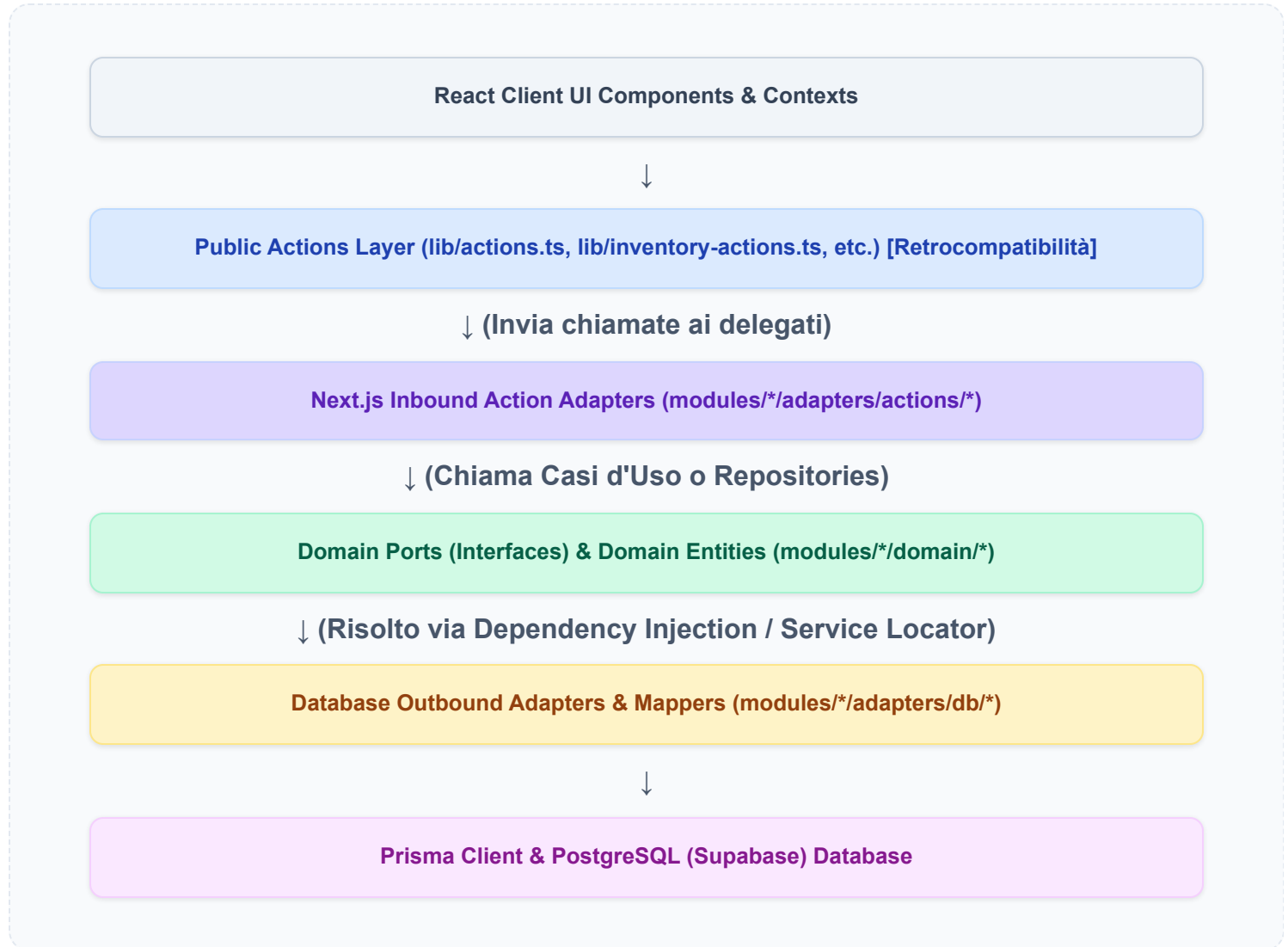
PROCESS

Modulo Processo

- **Entità pure:** Project, ProjectTask, SopDocument (SOP), ProcessAnomaly, LineSimulation (simulazioni FPES), ProcessRecipe, QualityReading.
- **Logica:** Standard Operating Procedures, Lean Manufacturing, bilanciamento celle, integrazione IA per consigli industriali.

2. Flusso dei Dati e Connessioni (Architettura Esagonale)

L'architettura garantisce che il **Core di Dominio** sia isolato da dettagli tecnologici esterni come database (Prisma, PostgreSQL) e framework web (Next.js Server Actions). Le frecce rappresentano la direzione delle chiamate:



3. Tecnologie Utilizzate

Livello	Tecnologia	Scopo nell'Applicazione
Framework Applicativo	Next.js 16 (App Router)	Routing, rendering server-side e Server Actions per le chiamate API del client.
Linguaggio	TypeScript 5	Tipizzazione statica e garanzia di correttezza a tempo di compilazione.
ORM / Database	Prisma & PostgreSQL (Supabase)	Persistenza dei dati, migrazioni automatiche e query tipizzate.
Integrazione IA	OpenAI API & Copilot	Analisi predittiva dei consumi, RCA automatico su anomalie e chatbot Lean.
Testing	Vitest	Unit testing delle entità e dei casi d'uso con repository mockati.
UI & UI Styling	Tailwind CSS & Radix UI	Componenti grafici moderni, accessibili e responsive.

4. Dettaglio Conessioni e Dependency Injection

Per mantenere i moduli disaccoppiati e consentire l'hot-reloading in ambiente Next.js serverless:

- **Service Locator:** Un registro centralizzato (ServiceLocator) memorizza le istanze concrete dei repository (ad es. PrismaWorkOrderRepository sotto la chiave IWorkOrderRepository).
- **Lazy Registry Registration:** Le registrazioni delle dipendenze vengono attivate all'importazione degli inbound adapters tramite blocchi try-catch, evitando errori di inizializzazione multipla e accoppiamenti rigidi all'avvio.
- **Event Bus Condiviso:** I moduli comunicano asincronamente tramite eventi registrati (ad es. quando si crea una WorkOrder da un piano di manutenzione preventiva).